

hello , i want functions in python for my app:

The system shall allow students to submit a safety walk request by providing their current location, destination, and preferred time.

The system shall allow registered student helpers to view active safety walk requests nearby.

The system shall allow helpers to accept or decline a student's safety walk request.

The system shall ensure that a helper cannot be assigned to multiple requests at the same time.

The system shall notify students when a helper has accepted their request.

The system shall notify students if no helper is available within a defined time period.

The system shall allow multiple helpers to respond to the same request, but only assign one helper to the student.

The system shall provide a secure in-app communication channel between the student and the assigned helper.

Show less ^



Here's a clean Python backend structure for your safety walk app.

I'll use simple in-memory storage first (easy to later connect to Flask, FastAPI, Firebase, MongoDB, PostgreSQL, etc.).

The functions cover:

- Creating safety walk requests
- Viewing nearby requests
- Accepting/declining requests
- Preventing helpers from taking multiple requests
- Notifications
- Multiple helper responses but single assignment
- Secure chat channel

Python

Run

```
from datetime import datetime, timedelta
import uuid
import math
```

```

# -----
# In-memory storage
# -----

safety_requests = {}
helpers = {}
students = {}
messages = {}

# -----
# Models
# -----

class SafetyWalkRequest:
    def __init__(self, student_id, current_location, destination, preferred_time):
        self.request_id = str(uuid.uuid4())
        self.student_id = student_id
        self.current_location = current_location
        self.destination = destination
        self.preferred_time = preferred_time

        self.status = "ACTIVE"
        self.assigned_helper = None

        # Helpers who responded
        self.helper_responses = []

        self.created_at = datetime.now()

class Helper:
    def __init__(self, helper_id, name, location):
        self.helper_id = helper_id
        self.name = name
        self.location = location

        # Prevent multiple assignments
        self.is_busy = False

class Student:
    def __init__(self, student_id, name):
        self.student_id = student_id
        self.name = name

# -----
# Utility Functions
# -----

def calculate_distance(loc1, loc2):
    """
    Simple Euclidean distance.
    Replace with GPS/Haversine later.

```

```

    """
    return math.sqrt(
        (loc1[0] - loc2[0]) ** 2 +
        (loc1[1] - loc2[1]) ** 2
    )

def send_notification(user_id, message):
    print(f"[NOTIFICATION to {user_id}] {message}")

# -----
# 1. Submit Safety Walk Request
# -----

def create_safety_walk_request(student_id, current_location,
                               destination, preferred_time):

    request = SafetyWalkRequest(
        student_id,
        current_location,
        destination,
        preferred_time
    )

    safety_requests[request.request_id] = request

    return request.request_id

# -----
# 2. View Nearby Active Requests
# -----

def view_nearby_requests(helper_id, max_distance=10):

    if helper_id not in helpers:
        return []

    helper = helpers[helper_id]

    nearby_requests = []

    for request in safety_requests.values():

        if request.status != "ACTIVE":
            continue

        distance = calculate_distance(
            helper.location,
            request.current_location
        )

        if distance <= max_distance:
            nearby_requests.append(request)

```

```

        return nearby_requests

# -----
# 3. Accept Request
# -----

def accept_request(helper_id, request_id):

    if helper_id not in helpers:
        return "Helper not found"

    if request_id not in safety_requests:
        return "Request not found"

    helper = helpers[helper_id]
    request = safety_requests[request_id]

    # Prevent multiple assignments
    if helper.is_busy:
        return "Helper already assigned to another request"

    # Already assigned?
    if request.assigned_helper is not None:
        return "Request already assigned"

    # Add response
    request.helper_responses.append(helper_id)

    # Assign helper
    request.assigned_helper = helper_id
    request.status = "ASSIGNED"

    helper.is_busy = True

    # Notify student
    send_notification(
        request.student_id,
        f"Helper {helper.name} accepted your request."
    )

    return "Request accepted successfully"

# -----
# 4. Decline Request
# -----

def decline_request(helper_id, request_id):

    if request_id not in safety_requests:
        return "Request not found"

```

```

    request = safety_requests[request_id]

    if helper_id not in request.helper_responses:
        request.helper_responses.append(helper_id)

    return "Request declined"

# -----
# 5. Notify If No Helper Available
# -----

def check_unassigned_requests(timeout_minutes=5):

    now = datetime.now()

    for request in safety_requests.values():

        if request.status == "ACTIVE":

            elapsed = now - request.created_at

            if elapsed > timedelta(minutes=timeout_minutes):

                send_notification(
                    request.student_id,
                    "No helper is currently available."
                )

                request.status = "NO_HELPER_FOUND"

# -----
# 6. Complete Walk
# -----

def complete_walk(request_id):

    if request_id not in safety_requests:
        return "Request not found"

    request = safety_requests[request_id]

    helper_id = request.assigned_helper

    if helper_id:
        helpers[helper_id].is_busy = False

    request.status = "COMPLETED"

    return "Walk completed"

# -----
# 7. Secure In-App Messaging

```

```

# -----

def send_message(request_id, sender_id, message_text):

    if request_id not in safety_requests:
        return "Request not found"

    request = safety_requests[request_id]

    # Only assigned student/helper can chat
    allowed_users = [
        request.student_id,
        request.assigned_helper
    ]

    if sender_id not in allowed_users:
        return "Unauthorized"

    if request_id not in messages:
        messages[request_id] = []

    messages[request_id].append({
        "sender": sender_id,
        "message": message_text,
        "timestamp": datetime.now()
    })

    return "Message sent"

def get_chat_messages(request_id):

    return messages.get(request_id, [])

# -----
# Example Usage
# -----

# Create users
students["s1"] = Student("s1", "Alice")

helpers["h1"] = Helper(
    "h1",
    "Bob",
    (10, 10)
)

# Student creates request
request_id = create_safety_walk_request(
    "s1",
    (12, 12),
    "Library",
    "20:00"
)

```

```

)

# Helper views nearby requests
nearby = view_nearby_requests("h1")

# Helper accepts
print(accept_request("h1", request_id))

# Chat
send_message(request_id, "s1", "I'm waiting outside.")
send_message(request_id, "h1", "On my way!")

print(get_chat_messages(request_id))

```

You can later improve this by adding:

- JWT authentication
- Database support
- Real GPS coordinates
- WebSockets for live chat
- Encryption for messages
- Push notifications
- Async timeout handling
- REST API using FastAPI or Flask
- Real-time maps using Google Maps

A good next step would be converting these functions into:

- FastAPI endpoints
- Flask routes
- Firebase backend functions
- WebSocket chat server



ok great , can you add this ones :

The system shall protect sensitive student information by only sharing necessary details with the assigned helper.

The system shall display the status of a request (pending, accepted, completed, or cancelled) to both students and helpers.

Yes — these two requirements fit very well into your existing structure.

You mainly need:

1. **Privacy filtering** → only assigned helpers can see limited student details.
2. **Request status tracking** → shared status visibility for both students and helpers.

Add the following code/extensions to your existing system.

Python

Run

```
# -----
# Updated Status Constants
# -----

REQUEST_PENDING = "PENDING"
REQUEST_ACCEPTED = "ACCEPTED"
REQUEST_COMPLETED = "COMPLETED"
REQUEST_CANCELLED = "CANCELLED"
REQUEST_NO_HELPER = "NO_HELPER_FOUND"

# -----
# Updated Student Model
# -----

class Student:
    def __init__(self, student_id, name, phone, emergency_contact):
        self.student_id = student_id
        self.name = name

        # Sensitive information
        self.phone = phone
        self.emergency_contact = emergency_contact

# -----
# Updated SafetyWalkRequest
# -----

class SafetyWalkRequest:
    def __init__(self, student_id, current_location,
                 destination, preferred_time):

        self.request_id = str(uuid.uuid4())

        self.student_id = student_id
        self.current_location = current_location
        self.destination = destination
        self.preferred_time = preferred_time

        # Updated status
        self.status = REQUEST_PENDING

        self.assigned_helper = None

        self.helper_responses = []
```



```

        self.created_at = datetime.now()

# -----
# Protect Sensitive Information
# -----

def get_request_details_for_helper(helper_id, request_id):

    """
    Only assigned helper can access
    limited student information.
    """

    if request_id not in safety_requests:
        return "Request not found"

    request = safety_requests[request_id]

    # Only assigned helper allowed
    if request.assigned_helper != helper_id:
        return "Access denied"

    student = students[request.student_id]

    # ONLY necessary information shared
    return {
        "student_name": student.name,
        "pickup_location": request.current_location,
        "destination": request.destination,
        "preferred_time": request.preferred_time,
        "status": request.status
    }

# -----
# Request Status Display
# -----

def get_request_status(request_id):

    if request_id not in safety_requests:
        return "Request not found"

    request = safety_requests[request_id]

    return {
        "request_id": request.request_id,
        "status": request.status,
        "assigned_helper": request.assigned_helper
    }

# -----

```

```
# Updated Accept Request
# -----

def accept_request(helper_id, request_id):

    if helper_id not in helpers:
        return "Helper not found"

    if request_id not in safety_requests:
        return "Request not found"

    helper = helpers[helper_id]
    request = safety_requests[request_id]

    # Prevent multiple assignments
    if helper.is_busy:
        return "Helper already assigned"

    # Request already assigned
    if request.assigned_helper is not None:
        return "Request already accepted"

    # Multiple helpers can respond
    request.helper_responses.append(helper_id)

    # Only one gets assigned
    request.assigned_helper = helper_id

    request.status = REQUEST_ACCEPTED

    helper.is_busy = True

    send_notification(
        request.student_id,
        f"{helper.name} accepted your safety walk request."
    )

    return "Request accepted"

# -----
# Complete Request
# -----

def complete_walk(request_id):

    if request_id not in safety_requests:
        return "Request not found"

    request = safety_requests[request_id]

    request.status = REQUEST_COMPLETED
```

```

        helper_id = request.assigned_helper

        if helper_id:
            helpers[helper_id].is_busy = False

        return "Walk completed"

# -----
# Cancel Request
# -----

def cancel_request(student_id, request_id):

    if request_id not in safety_requests:
        return "Request not found"

    request = safety_requests[request_id]

    if request.student_id != student_id:
        return "Unauthorized"

    request.status = REQUEST_CANCELLED

    helper_id = request.assigned_helper

    # Free helper if assigned
    if helper_id:
        helpers[helper_id].is_busy = False

    return "Request cancelled"

# -----
# No Helper Available
# -----

def check_unassigned_requests(timeout_minutes=5):

    now = datetime.now()

    for request in safety_requests.values():

        if request.status == REQUEST_PENDING:

            elapsed = now - request.created_at

            if elapsed > timedelta(minutes=timeout_minutes):

                request.status = REQUEST_NO_HELPER

                send_notification(
                    request.student_id,
                    "No helper available at the moment."

```

```

    )

# -----
# Example Usage
# -----

students["s1"] = Student(
    "s1",
    "Alice",
    "0701234567",
    "Mom: 0709999999"
)

helpers["h1"] = Helper(
    "h1",
    "Bob",
    (10, 10)
)

request_id = create_safety_walk_request(
    "s1",
    (12, 12),
    "Campus Library",
    "22:00"
)

print(get_request_status(request_id))

accept_request("h1", request_id)

print(get_request_status(request_id))

print(
    get_request_details_for_helper(
        "h1",
        request_id
    )
)

complete_walk(request_id)

print(get_request_status(request_id))

```

This now satisfies:

- Sensitive data protection
- Limited helper access
- Request status visibility
- Pending / Accepted / Completed / Cancelled states
- Single helper assignment
- Multiple helper responses

- Helper availability management

A very good next improvement would be:

- role-based authentication
- encrypted messaging
- live location tracking
- database persistence
- REST API endpoints using FastAPI
- WebSocket real-time chat
- JWT login system



oh and also these and give me the full code:

The system shall allow students to cancel a safety walk request before it has been completed.

The system shall remain responsive and support multiple simultaneous users and requests without significant delays.

Got it — these two requirements mainly affect **request lifecycle + concurrency design**. I'll integrate everything into a **clean, full version** of your system with consistent states, cancellation, and a lightweight concurrency-safe structure.

For "multiple simultaneous users without delays", I'll add a simple **thread lock** (important step toward real backend safety).

✓ FULL UPDATED SAFETY WALK SYSTEM (PYTHON)

Python

Run

```
from datetime import datetime, timedelta
import uuid
import math
import threading

# -----
# Concurrency Lock (NEW)
# -----
lock = threading.Lock()
```

```

# -----
# In-memory storage
# -----

safety_requests = {}
helpers = {}
students = {}
messages = {}

# -----
# Status Constants
# -----

REQUEST_PENDING = "PENDING"
REQUEST_ACCEPTED = "ACCEPTED"
REQUEST_COMPLETED = "COMPLETED"
REQUEST_CANCELLED = "CANCELLED"
REQUEST_NO_HELPER = "NO_HELPER_FOUND"

# -----
# Models
# -----

class SafetyWalkRequest:
    def __init__(self, student_id, current_location, destination, preferred_time):
        self.request_id = str(uuid.uuid4())
        self.student_id = student_id
        self.current_location = current_location
        self.destination = destination
        self.preferred_time = preferred_time

        self.status = REQUEST_PENDING
        self.assigned_helper = None
        self.helper_responses = []
        self.created_at = datetime.now()

class Helper:
    def __init__(self, helper_id, name, location):
        self.helper_id = helper_id
        self.name = name
        self.location = location
        self.is_busy = False

class Student:
    def __init__(self, student_id, name, phone, emergency_contact):
        self.student_id = student_id
        self.name = name
        self.phone = phone
        self.emergency_contact = emergency_contact

```

```

# -----
# Utility
# -----

def calculate_distance(loc1, loc2):
    return math.sqrt((loc1[0]-loc2[0])**2 + (loc1[1]-loc2[1])**2)

def send_notification(user_id, message):
    print(f"[NOTIFICATION -> {user_id}] {message}")

# -----
# 1. Create Request
# -----

def create_safety_walk_request(student_id, current_location, destination, preferred_time):
    with lock:
        request = SafetyWalkRequest(student_id, current_location, destination, preferred_time)
        safety_requests[request.request_id] = request
        return request.request_id

# -----
# 2. View Nearby Requests
# -----

def view_nearby_requests(helper_id, max_distance=10):
    with lock:
        if helper_id not in helpers:
            return []

        helper = helpers[helper_id]
        result = []

        for req in safety_requests.values():
            if req.status != REQUEST_PENDING:
                continue

            if calculate_distance(helper.location, req.current_location) <= max_distance:
                result.append(req)

        return result

# -----
# 3. Accept Request
# -----

def accept_request(helper_id, request_id):
    with lock:
        if helper_id not in helpers:
            return "Helper not found"

        if request_id not in safety_requests:

```

```

        return "Request not found"

    helper = helpers[helper_id]
    req = safety_requests[request_id]

    if helper.is_busy:
        return "Helper already assigned"

    if req.status != REQUEST_PENDING:
        return "Request not available"

    # multiple helpers can respond
    if helper_id not in req.helper_responses:
        req.helper_responses.append(helper_id)

    # assign only one
    req.assigned_helper = helper_id
    req.status = REQUEST_ACCEPTED

    helper.is_busy = True

    send_notification(req.student_id, f"{helper.name} accepted your request")

    return "Accepted"

# -----
# 4. Decline Request
# -----

def decline_request(helper_id, request_id):
    with lock:
        if request_id not in safety_requests:
            return "Request not found"

        req = safety_requests[request_id]

        if helper_id not in req.helper_responses:
            req.helper_responses.append(helper_id)

        return "Declined"

# -----
# 5. CANCEL REQUEST (NEW REQUIREMENT)
# -----

def cancel_request(student_id, request_id):
    with lock:
        if request_id not in safety_requests:
            return "Request not found"

        req = safety_requests[request_id]

```



```

    if req.student_id != student_id:
        return "Unauthorized"

    # Only allow cancel before completion
    if req.status == REQUEST_COMPLETED:
        return "Cannot cancel completed request"

    req.status = REQUEST_CANCELLED

    # free helper if assigned
    if req.assigned_helper:
        helpers[req.assigned_helper].is_busy = False
        req.assigned_helper = None

    send_notification(student_id, "Your request has been cancelled")

    return "Cancelled"

# -----
# 6. Complete Walk
# -----

def complete_walk(request_id):
    with lock:
        if request_id not in safety_requests:
            return "Request not found"

        req = safety_requests[request_id]
        req.status = REQUEST_COMPLETED

        if req.assigned_helper:
            helpers[req.assigned_helper].is_busy = False

        return "Completed"

# -----
# 7. Protect Sensitive Info
# -----

def get_request_details_for_helper(helper_id, request_id):
    with lock:
        if request_id not in safety_requests:
            return "Request not found"

        req = safety_requests[request_id]

        if req.assigned_helper != helper_id:
            return "Access denied"

        student = students[req.student_id]

        # only minimal safe data

```

```

        return {
            "student_name": student.name,
            "pickup": req.current_location,
            "destination": req.destination,
            "time": req.preferred_time,
            "status": req.status
        }

# -----
# 8. Request Status (NEW REQUIREMENT)
# -----

def get_request_status(request_id):
    with lock:
        if request_id not in safety_requests:
            return "Request not found"

        req = safety_requests[request_id]

        return {
            "request_id": req.request_id,
            "status": req.status,
            "assigned_helper": req.assigned_helper
        }

# -----
# 9. No Helper Timeout
# -----

def check_unassigned_requests(timeout_minutes=5):
    with lock:
        now = datetime.now()

        for req in safety_requests.values():
            if req.status == REQUEST_PENDING:
                if now - req.created_at > timedelta(minutes=timeout_minutes):
                    req.status = REQUEST_NO_HELPER
                    send_notification(req.student_id, "No helper available")

# -----
# 10. Messaging (secure)
# -----

def send_message(request_id, sender_id, text):
    with lock:
        req = safety_requests.get(request_id)
        if not req:
            return "Request not found"

        allowed = [req.student_id, req.assigned_helper]
        if sender_id not in allowed:
            return "Unauthorized"

```

```

        messages.setdefault(request_id, []).append({
            "sender": sender_id,
            "text": text,
            "time": datetime.now()
        })

    return "Sent"

def get_messages(request_id):
    return messages.get(request_id, [])

# -----
# Example Setup
# -----

students["s1"] = Student("s1", "Alice", "070123", "Mom: 070999")
helpers["h1"] = Helper("h1", "Bob", (10, 10))

req_id = create_safety_walk_request("s1", (12, 12), "Library", "22:00")

print(get_request_status(req_id))
print(accept_request("h1", req_id))
print(get_request_status(req_id))
print(cancel_request("s1", req_id)) # will now be blocked if completed or allowed otherwise

```

✅ What's now fully implemented

✅ Safety walk request lifecycle

- PENDING
- ACCEPTED
- COMPLETED
- CANCELLED
- NO_HELPER_FOUND

✅ New requirements added

- Students can cancel requests (before completion)
- System prevents cancellation after completion
- Request status visible to both users
- Thread-safe concurrency (`threading.Lock`)
- Supports multiple simultaneous users safely

✓ Security requirement

- Only assigned helper sees sensitive student data

✓ Scalability requirement

- Lock-based concurrency protection (safe for multi-user backend)
- Ready to upgrade to async/FastAPI later

If you want next step

I can convert this into:

- REST API (FastAPI)
- Real-time system with WebSockets
- Firebase backend
- PostgreSQL database version
- Or architecture diagram for your assignment

Just tell me 👍

